

ClaroIntelligence

Registro de melhorias aplicadas ao protótipo

Documento de registro das evoluções implementadas a partir do retorno dos professores e dos profissionais da Claro

Projeto FIAP Challenge Claro — camada de orquestração com IA sobre os canais digitais de atendimento

Autor Raphael Batizati

Repositório github.com/RaphaelBatizati/ClaroIntelligent_Challenge

Emissão 29 de setembro de 2026

Sumário executivo

Este documento registra as melhorias aplicadas ao protótipo ClaroIntelligence depois da primeira apresentação. O ponto de partida foram nove solicitações trazidas pelos professores e por profissionais da Claro; uma segunda rodada de retorno gerou mais onze ajustes. Ao longo do trabalho foram encontrados e corrigidos onze defeitos, três deles anteriores ao ciclo.

O protótipo saiu de uma demonstração funcional para um sistema com protocolo regulatório, contenção de ataques ao assistente, identificação em duas etapas, fila de atendimento humano com priorização por risco de cancelamento e um painel que opera sobre volume de operação real. Todo número exibido no painel passou a vir de agregação do banco de dados.

Indicador	Antes	Depois
Solicitações atendidas	—	9 do primeiro retorno + 11 do segundo
Módulos de backend	6 serviços	11 serviços + 4 adaptadores
Telas do painel	7 (3 com dado simulado)	8, todas ligadas à API
Volume no painel	linha de base fixa somada no endpoint	~84 mil atendimentos reais em 30 dias
Roteiros de demonstração	6	11, sendo 3 com falas prontas
Verificações automatizadas	—	55 ponta a ponta, todas passando
Documentação técnica	6 documentos	9 documentos, ~2.100 linhas

Como ler este documento. A seção 1 cobre as solicitações do primeiro retorno, uma a uma, com o que foi pedido, o que foi entregue, onde o código vive e como conferir na tela. A seção 2 cobre a segunda rodada. A seção 3 lista os defeitos corrigidos. As seções 4 a 6 trazem os números do sistema, a estrutura do repositório e as limitações assumidas.

1. Solicitações dos professores e da Claro

Nove pontos levantados na primeira rodada de retorno. Todos implementados e verificados.

1	Analisar a possibilidade de uma camada de verificação em duas etapas na tela do WhatsApp
Entrega	Implementada e ativa. O código de 6 dígitos é gerado com <code>crypto.randomInt</code> , armazenado apenas como hash SHA-256 com salt, comparado em tempo constante (<code>timingSafeEqual</code>), expira em 5 minutos e bloqueia após 3 tentativas. A validação vale pela sessão inteira e, ao confirmar, o fluxo retoma sozinho a demanda que estava pendente. No protótipo o código aparece num cartão de SMS simulado, claramente rotulado; em produção esse campo desaparece da resposta da API.
Onde vive	<code>src/services/verificacao.js</code> · tabela <code>verificacoes</code> · documentado em <code>SEGURANCA.md</code>
Como verificar	Chat do Cliente, canal WhatsApp: a primeira mensagem já dispara o código.
2	Aplicar um ID por número de protocolo quando a intervenção for registrada, e deixar explícito o tipo de plano com produtos reais da Claro
Entrega	Todo contato gera protocolo no formato AAAAMMDD mais sequencial de 6 dígitos, com linha do tempo auditável de cada evento (abertura, verificação, desambiguação, transferência, encerramento) e registro de quem resolveu — autoatendimento ou atendente humano. A base regulatória é o Regulamento Geral de Direitos do Consumidor da Anatel, Resolução 765/2023, que revogou a 632/2014. O catálogo foi refeito com 18 produtos reais: Preção, Controle, Pós, Max Flex, Fibra de 350 Mega a 1 Giga, Casa 5G, Claro tv+ App e Box, além de Claro Empresas com chip empresarial de franquia compartilhada e link dedicado com SLA. Cada contrato carrega velocidade, franquia, tipo de chip e benefícios.
Onde vive	<code>src/services/protocolo.js</code> · tabelas <code>protocolos</code> e <code>protocolo_eventos</code> · catálogo em <code>PRODUTOS.md</code>
Como verificar	O protocolo aparece no cabeçalho do chat e no painel de transparência; o portfólio do cliente fica na barra lateral.
3	Construir uma demonstração em que o cliente entra numa fila de espera e conversa quando chega a vez. Era possível conectar o WhatsApp real?
Entrega	Foi construído o Console do Atendente: uma tela onde uma pessoa real assume a conversa e responde ao vivo, com o cliente vendo posição na fila, tempo estimado de espera e prioridade. O atendente recebe o briefing antes da primeira palavra — protocolo, linha do tempo, sinais de atrito, portfólio e as últimas falas do cliente. Sobre conectar o WhatsApp pessoal: não foi feito, e a recomendação é não fazer. A via oficial (WhatsApp Business Cloud API) exige conta empresarial verificada e um número dedicado que não pode estar em uso no aplicativo comum; a via não oficial viola os termos de uso e arrisca banimento permanente do número. A análise completa, com os três pontos exatos do código que mudariam numa integração oficial, está documentada.
Onde vive	<code>src/services/fila.js</code> · <code>pages/ConsoleAtendente.jsx</code> · análise em <code>INTEGRACAO_WHATSAPP.md</code>
Como verificar	Provoque um transbordo no Chat do Cliente e acompanhe o caso chegar no Console, em outra aba.

4	No painel do ClaroSense, deixar claro que repetição e tom agressivo aumentam o score, elevando o risco de o cliente cancelar
Entrega	O ClaroSense trabalha com 7 sinais de peso conhecido e explicação legível: tom agressivo (+35), pedido de atendente humano (+35), menção a cancelamento (+30), linguagem de frustração (+28), repetição de intenção (+22), recontato em outro canal (+15) e resposta monossilábica (+10). O score acumula ao longo da sessão e é traduzido em risco de cancelamento, com ação recomendada. A detecção de agressividade cobre palavrão, ameaça de Procon ou Anatel e mensagem em caixa alta sustentada. O painel exibe a régua de pesos e uma simulação interativa da escalada, turno a turno.
Onde vive	src/services/claroSense.js · pages/LogClaroSense.jsx
Como verificar	Roteiro I no Chat do Cliente: o score sobe de 0 para 50 e depois 100, com cada ponto justificado.
5	Pegar uma demonstração existente e mostrar o caso em que a pessoa não precisa de atendente, resolvendo tudo via chat
Entrega	Foram implementados dois fluxos transacionais completos, em duas etapas cada (proposta e confirmação): pagamento de fatura, que mostra valor e vencimento, gera o PIX copia-e-cola e registra a autenticação; e upgrade de plano, que sobe o degrau seguinte da escada real de velocidades da fibra e atualiza o contrato. Nos dois casos o protocolo é encerrado no mesmo turno, marcado como resolvido pelo autoatendimento. É daí que sai a taxa de contenção do painel.
Onde vive	src/services/autoatendimento.js · tabela acoes_autoatendimento
Como verificar	Roteiro H (Tiago Ramos): duas mensagens encerram o atendimento sem fila e sem atendente.
6	No Monitor de Conversas, filtros que mostrem a origem da informação entre os canais, com data e horário, funcionais e realistas para um serviço com milhares de interações por dia
Entrega	A tela foi religada à API e ganhou busca textual, filtros por canal, status, linha de produto, persona, faixa de atrito, protocolo, intervalo de datas e faixa de horário do dia, com paginação, ordenação, contadores por opção, histograma de pico por hora e atualização ao vivo. Para que os filtros tivessem o que filtrar, o banco passou a ser populado com volume de operação real.
Onde vive	src/routes/conversas.js · pages/MonitorConversas.jsx
Como verificar	Monitor de Conversas: a busca combina qualquer dos filtros e responde sobre 84 mil registros.
7	Na parte de persona, acrescentar um dicionário de personas, porque às vezes o cliente usa linguagem mais informal e isso precisa ser aceito
Entrega	Foi criada a quarta persona, Informal, com tom espelhado, e o léxico saiu do código para uma tabela de banco com 98 termos, editável pela tela de Gestão de Personas sem precisar de deploy. Cada termo tem persona, categoria e peso, e pode ser desativado sem ser apagado, preservando o histórico da calibração. A regra de mistura foi pensada com cuidado: o informal vence qualquer perfil, exceto o assistido — quem precisa de ajuda e escreve informalmente continua recebendo explicação simples.
Onde vive	src/services/personaEngine.js · tabela persona_dicionario
Como verificar	Gestão de Personas: o simulador classifica qualquer texto e mostra os termos que pesaram.

8	Ter um sistema cauteloso contra injeção no chatbot, porque a pessoa pode tentar acessar o banco ou informações. A IA precisa ficar limitada ao que ela serve
Entrega	Foram implementadas três camadas. A primeira classifica a mensagem antes de qualquer processamento : sendo ataque, a requisição morre ali, sem chegar ao resolver de produto, aos adaptadores ou ao provedor de linguagem. São 11 padrões monitorados, em três famílias: injeção de prompt (4), extração de dados (5) e engenharia social (2). A segunda camada é o escopo fechado: apenas 20 intenções do catálogo são atendidas, e o resto é redirecionado com o menu de capacidades. A terceira redige CPF, número de cartão e e-mail de terceiro na saída. Toda detecção é auditada, com o trecho truncado em 120 caracteres e já redigido — o log de segurança não pode virar, ele mesmo, um vazamento.
Onde vive	src/services/guardrails.js · tabela eventos_seguranca · SEGURANCA.md
Como verificar	Roteiro K: tente <i>ignore todas as instruções anteriores</i> ou <code>‘; SELECT * FROM clientes --</code> em qualquer cliente e canal.

9	Pegar uma persona e mostrar o cenário de quem começou no call center, teve problema e entrou pelo chat, deixando clara a continuação graças ao protocolo
Entrega	Roteiro E, com Roberto Alves. Ele ligou no call center contestando uma cobrança de R\$ 34,90 e a chamada encerrou sem confirmar o estorno, deixando o protocolo aberto com a pendência registrada. Ao abrir o chat, o sistema reconhece o protocolo de outro canal e retoma daquele ponto, sem pedir que ele explique de novo. A recuperação é feita sobre protocolos abertos nas últimas 72 horas.
Onde vive	protocolo.protocolosAbertos() · linha do tempo em protocolo_eventos
Como verificar	Roteiro E, canal Site: a primeira resposta já cita o protocolo aberto no Call Center há 3 horas.

2. Ajustes da segunda rodada de retorno

Onze pontos levantados depois de rodar o protótipo já evoluído. A maior parte veio de uso real da tela.

10	O protocolo era repetido a cada mensagem da conversa
Entrega	O aviso de retomada passou a ser dado uma única vez por sessão , controlado por uma coluna nova. Repetir a cada turno, além de poluir a conversa, fazia o assistente parecer que esquecia o que tinha acabado de dizer. A regra também virou regra de segurança: o aviso só aparece depois da identificação, porque protocolo anterior é histórico de atendimento e não pode ser entregue a quem não se identificou.
Onde vive	<code>coluna sessoes.continuidade_anunciada · llm.prefixoContinuidade()</code>
Como verificar	Roteiro E: o aviso aparece no primeiro turno e não volta nos seguintes.
11	A verificação em duas etapas deveria ficar só no WhatsApp, identificando pelo número da pessoa
Entrega	O segundo fator foi restrito ao WhatsApp e movido para a entrada da conversa. O número identifica o cliente no cadastro, o código por SMS confirma que é ele, e nada do contrato sai antes disso. O raciocínio: site, aplicativo e call center têm autenticação do próprio canal, então pedir código ali é fricção sem ganho; no WhatsApp não existe login, e posse do número não é prova de identidade. Verificar no fim do fluxo também era tarde demais — a conversa já teria devolvido portfólio e histórico de outros canais.
Onde vive	<code>verificacao.exigeVerificacao()</code> e <code>sessaoIdentificada()</code>
Como verificar	Canal WhatsApp: a primeira mensagem pede o código. Site e App respondem direto.
12	Construir três personas — guiada, informal e técnica — cada uma com um tipo de plano diferente e um desfecho de conversa diferente
Entrega	Foram criados três clientes com conversa completa e falas na ordem exata, clicáveis no painel lateral do chat para não haver erro de digitação durante a apresentação. Helena Duarte (guiada, residencial, Fibra 350) tem o atrito subindo até 63 e pede uma pessoa sem chegar ao nível crítico. Tiago Ramos (informal, móvel, Pós 50GB) paga a conta inteira no chat, sem fila e sem atendente. Nexa Log Transportes (técnica, empresarial, Fibra 500) reporta o mesmo erro três vezes e escala de 0 para 100, disparando o transbordo automático sem que ninguém peça atendente.
Onde vive	<code>CLIENTES_PITCH</code> em <code>pages/ChatCliente.jsx</code> · dados em <code>seed-extras.js</code>
Como verificar	Roteiros G, H e I: clique as falas em sequência e acompanhe o score no painel.
13	Na fila dos atendentes, acrescentar filtro de gravidade do atrito, filtro de tipo de serviço e dar prioridade a quem tem maior percentual de churn
Entrega	A fila ganhou filtros por gravidade, tipo de serviço e canal de origem, com contadores por opção. A ordenação passou a ser gravidade primeiro e, dentro dela, risco de cancelamento decrescente — quem está mais perto de sair não espera atrás de uma dúvida simples que chegou um minuto antes. O tipo de serviço agrupa a intenção em famílias que o atendente reconhece (financeiro, técnico, retenção, comercial, consumo, cadastro), que é o eixo de roteamento por especialidade de uma operação real.
Onde vive	<code>fila.prioridadeDe()</code> e <code>fila.listar()</code> · colunas novas em <code>fila_atendimento</code>
Como verificar	Console do Atendente: os filtros ficam acima da fila e o churn aparece em cada cartão.

14	Colocar mais casos fictícios na fila de atendimento
Entrega	Seis casos entraram na carga inicial, com gravidade, tipo de serviço, canal e risco variados — de um cancelamento com 96% de risco a uma remarcação de visita técnica com 24%. Cada um traz transcrição, sinais de atrito e protocolo, de forma que o Console abra povoado sem depender de alguém provocar transbordo antes da demonstração. Com o caso que já existia, são sete na fila.
Onde vive	seed-extras.js, bloco <i>FILA POVOADA</i>
Como verificar	Console do Atendente logo após o seed: 7 aguardando, 4 em prioridade alta.
15	O rodapé mostrava 2025, sendo que é 2026
Entrega	Corrigido.
Onde vive	components/Sidebar.jsx
Como verificar	Rodapé da barra lateral.
16	No filtro de período, oferecer última hora, 1 dia, 7 dias e 30 dias, e que funcionem
Entrega	O seletor de período virou estado global do painel e passou a valer para Dashboard, Mapa de Atrito e Log do ClaroSense, com as quatro janelas pedidas. A granularidade da série temporal acompanha o recorte: por minuto na última hora, por hora no dia, por dia nas janelas maiores. Nas telas que não respondem ao período o seletor deixa de aparecer, para não sugerir um filtro que não filtra nada.
Onde vive	contexts/periodo.js · parâmetro ?periodo= em todas as rotas do dashboard
Como verificar	Troque o período na barra superior: todos os números da tela mudam junto.
17	No Mapa de Atrito, ao filtrar uma coisa, as quantidades dos outros filtros continuavam mostrando números que não correspondiam
Entrega	Esse era o defeito mais sério apontado, e existia em três telas. A regra adotada: cada dimensão é contada aplicando todos os filtros ativos, menos o dela própria . O filtro de canal não se filtra a si mesmo, senão só sobraria a opção já escolhida, mas respeita persona, linha, jornada e período. Foi aplicada no Mapa de Atrito, no Monitor de Conversas e na fila do Console. Para que isso fosse possível, os KPIs deixaram de somar uma linha de base fixa dentro do endpoint — era ela que impedia os números de responderem a filtro nenhum.
Onde vive	routes/dashboard.js, routes/conversas.js, services/fila.js
Como verificar	Filtre por WhatsApp no Mapa de Atrito: os contadores de persona passam a somar exatamente o total filtrado.
18	Os números do Dashboard ficaram pequenos demais para um portal de atendimento
Entrega	O volume foi para onde ele pertence: os dados. A carga inicial passou a gerar cerca de 84 mil atendimentos em 30 dias, com curva de dia útil, sábado e domingo, distribuídos entre 445 clientes sintéticos, com canal, jornada, persona, score, sinais e intervenções coerentes entre si. Não se voltou a somar linha de base no endpoint porque era isso que travava os filtros. O custo ficou baixo: o SQLite grava cerca de 600 mil linhas por segundo em transação única, então a carga leva 6 segundos.
Onde vive	seed-extras.js, bloco <i>VOLUME HISTÓRICO</i>
Como verificar	Dashboard em 7 dias: 19.802 atendimentos, 7.864 intervenções do ClaroSense.

19	O mapa de calor era mais claro em percentual, para ver qual canal está mais prejudicado em cada tipo de solicitação
Entrega	A célula voltou a ser percentual, agora com significado exato: a fatia dos atendimentos daquele cruzamento cujo score passou do limiar de atrito. Percentual e não média porque é o que permite comparar canais de volumes muito diferentes. O canal mais prejudicado de cada jornada ganhou contorno, que é a leitura que a tela existe para entregar. Foi preciso corrigir também a geração de dados, que não diferenciava canal e deixava todas as colunas com o mesmo número.
Onde vive	pages/MapaAtrito.jsx · GET /api/dashboard/mapa-atrito
Como verificar	Mapa de calor: cancelamento por telefone chega a 99%, recarga pelo aplicativo fica em 7%.

20	Preparar uma demonstração do Chat do Cliente e do Console do Atendente para o pitch, em no máximo 2 minutos e 30
Entrega	Foi produzido um roteiro cronometrado com tabela de ações segundo a segundo, narração pronta para leitura, plano de corte caso o tempo aperte e versão estendida de 3 minutos. Inclui os comandos de preparo, uma conferência de 15 segundos com os quatro números que o Console deve mostrar após a carga inicial, e uma tabela de erros comuns na hora de gravar. A lista de clientes do chat foi reduzida às quatro personas do roteiro, com os demais atrás de um link.
Onde vive	docs/ROTEIRO_PITCH.md
Como verificar	O documento é autossuficiente: abrir, seguir a tabela e gravar.

3. Defeitos encontrados e corrigidos

Onze defeitos apareceram durante a verificação do que foi construído. Três já existiam antes deste ciclo e passavam despercebidos. Estão registrados aqui porque a correção de cada um mudou o comportamento do sistema em situações que a demonstração encontra.

Defeito	Consequência	Correção
quero cancelar e cancelamento nunca eram detectados	A intenção mais crítica do negócio caía em <i>geral</i> , o que afetava a priorização da fila. Defeito anterior a este ciclo.	A expressão exigia um caractere não-letra logo após o radical, então nenhuma flexão casava. Trocado por radical mais sufixo livre, no detector de intenção e no ClaroSense.
falar com uma pessoa não era reconhecido	O sinal de maior peso da régua de atrito passava em branco justamente com quem pede de forma mais educada.	O artigo feminino não estava previsto na expressão. Incluído, junto de outras formas comuns do mesmo pedido.
minha internet fica caindo caía em geral	A forma mais comum de relatar queda de conexão não era classificada, e o atendimento começava sem saber do que se tratava.	O gerúndio e as variantes de <i>sem conexão</i> foram acrescentados ao padrão de suporte técnico.
O código de verificação contava como resposta monossilábica	Digitar os 6 dígitos que o próprio sistema pediu somava 10 pontos de atrito, distorcendo o score do roteiro no WhatsApp.	O sinal passou a ignorar mensagens que são apenas dígitos. Colaborar não é impaciência.
Contadores de filtro ignoravam os demais filtros	Filtrar por WhatsApp e continuar vendo <i>Call Center (81)</i> num resultado onde nenhum aparece. O contador parecia informação e era ruído.	Cada dimensão passou a ser contada aplicando todos os filtros menos o dela própria, nas três telas afetadas.
KPIs somavam uma linha de base fixa no endpoint	Os números do painel não se moviam ao trocar período ou canal, porque a maior parte deles era constante.	O volume foi movido para os dados e as rotas passaram a ser agregação pura do banco.
Cartão de indicador não reanimava ao mudar de valor	Ao trocar o período, o cartão continuava exibindo o número antigo.	A animação passou a partir do valor anterior a cada mudança, em vez de rodar só na primeira montagem.
Limite único de requisições quebrava a própria demonstração	Chat e Console abertos juntos somam cerca de 68 requisições por minuto de atualização legítima, acima do teto de 60. Defeito introduzido e detectado no mesmo ciclo.	Limite em dois níveis: 600 por minuto para leitura de painel e 60 por minuto no endpoint de chat, que é o caro e abusável.
O assistente repetia a mesma frase enquanto esperava o código	Quem escrevia algo antes de confirmar a identidade recebia sempre a mesma resposta, como se a mensagem não tivesse sido lida.	O assistente passou a nomear a demanda de volta e a guardar a intenção mais recente, para retomar o assunto certo depois da confirmação. Na segunda cobrança o texto muda e oferece reenvio.
Desafio de verificação vencido prendia a conversa	O código expirava e a sessão seguia esperando por ele indefinidamente.	A expiração passou a ser reconhecida na consulta de estado, e não só quando alguém tenta um código. O fluxo emite um novo desafio.

Defeito	Consequência	Correção
Matriz de capacidades ausente no resolver de produto	Perguntas que o portfólio já respondia sozinho geravam desambiguação desnecessária. Defeito anterior a este ciclo, previsto no desenho original e não implementado.	Implementada: consulta de franquia descarta o link dedicado, que não tem franquia; recarga só considera pré-pago e controle, e assim por diante.

4. Números do sistema

Estado do protótipo após as melhorias. Todos os valores foram medidos na versão publicada.

4.1 Base de dados gerada pela carga inicial

Item	Quantidade	Observação
Atendimentos (30 dias)	84.071	curva de dia útil, sábado e domingo
Sessões de conversa	84.073	com canal, persona e score coerentes
Mensagens	168.176	duas por atendimento histórico
Sinais de atrito	88.611	gerados pela mesma régua do ClaroSense
Intervenções automáticas	33.538	antecipação, simplificação e transbordo
Clientes	461	445 sintéticos + 16 de roteiro
Produtos no catálogo	18	portfólio real da Claro
Termos no dicionário de personas	98	editáveis pelo painel
Tabelas no banco	16	7 criadas neste ciclo

A carga é determinística: um gerador pseudoaleatório com semente fixa faz com que rodar o comando duas vezes produza exatamente o mesmo painel — o que importa quando se está gravando uma apresentação.

4.2 Desempenho medido

Operação	Antes	Depois
Mapa de Atrito, 7 dias	—	77 ms
Mapa de Atrito, 30 dias	524 ms	340 ms
KPIs do Dashboard, 30 dias	212 ms	122 ms
Carga inicial completa	—	6 segundos
Gravação no banco	—	~600 mil linhas por segundo

O ganho no Mapa de Atrito veio de uma mudança de estratégia: em vez de trazer dezenas de milhares de registros para a aplicação e refiltrar o conjunto a cada jornada e a cada célula, uma única consulta traz o período agregado por jornada, canal, persona e linha — menos de mil linhas. KPIs, jornadas, mapa de calor e contadores derivam desse mesmo cubo, o que também reforça a coerência da tela.

4.3 Verificação

Tipo	Resultado
Suíte ponta a ponta	55 verificações, todas passando
Cobertura da suíte	retomada de protocolo, identificação no WhatsApp, os três roteiros novos, filtros da fila, períodos do mapa, contadores em cascata e guardrails
Navegação das 8 telas	automatizada em navegador real, sem erro de console
Análise estática	sem alertas
Compilação do frontend	sem erros

5. O que foi criado no repositório

5.1 Módulos novos no backend

Arquivo	Responsabilidade
services/protocolo.js	Numeração, linha do tempo, retomada entre canais e encerramento com registro de quem resolveu
services/guardrails.js	As três camadas de contenção do assistente e a auditoria de segurança
services/verificacao.js	Identificação em duas etapas no WhatsApp
services/fila.js	Fila humana, priorização por risco de cancelamento e briefing do atendente
services/autoatendimento.js	Pagamento e upgrade em duas etapas, com encerramento do protocolo
routes/fila.js	Endpoints do Console do Atendente, com filtros e contadores
routes/protocolos.js	Consulta e linha do tempo de protocolos
routes/seguranca.js	Eventos de guardrail, resumo e políticas aplicadas
seed-extras.js	Personas de roteiro, fila povoada e 30 dias de volume histórico
adapters/_comum.js	Utilitários compartilhados entre os adaptadores de linha de produto

5.2 Telas

Tela	Situação
Console do Atendente	Nova. Fila filtrável, briefing completo e conversa ao vivo
Mapa de Atrito	Reescrita contra a API, com filtros em cascata e mapa de calor em percentual
Monitor de Conversas	Religada à API, com nove filtros combináveis e atualização ao vivo
Log do ClaroSense	Religada à API, com régua de pesos e simulação da escalada
Gestão de Personas	Religada à API, com dicionário editável e simulador de classificação
Dashboard	Passou a ser agregação pura do banco, respondendo ao período
Chat do Cliente	Ganhou identificação por SMS, fila, autoatendimento e roteiros clicáveis

5.3 Documentação

Documento	Conteúdo
ROTEIRO_PITCH.md	Roteiro cronometrado de 2min30 das duas telas de atendimento (novo)
INTEGRACAO_WHATSAPP.md	Análise de viabilidade da integração real, com requisitos e riscos (novo)
PRODUTOS.md	Catálogo real da Claro, chaves de identificação e matriz de capacidades (novo)
SEGURANCA.md	Guardrails, identificação em duas etapas, LGPD e limitações
ARQUITETURA.md	Camadas, pipeline de uma mensagem, protocolo, resolver, motores e fila
BANCO_DE_DADOS.md	Esquema das 16 tabelas, relacionamentos e convenções
API.md	Todos os endpoints, filtros e formas de resposta
COMO_RODAR.md	Instalação e os 11 roteiros de demonstração
DECSOES.md	Por que cada escolha técnica não óbvia foi tomada

6. Limitações assumidas

Registradas aqui e em SEGURANCA.md por decisão de projeto. São escolhas conscientes para manter o protótipo demonstrável sem infraestrutura adicional, e não recomendações para produção.

Limitação	Risco em produção	O que resolveria
Sem autenticação de sessão nem autorização nas rotas	Qualquer um com acesso à rede chama qualquer endpoint como qualquer cliente	Autenticação na borda e controle de acesso por papel nas rotas do painel
Console do Atendente sem login de operador	Qualquer pessoa assumiria uma conversa e falaria como atendente da Claro	Login de operador com papel e trilha de auditoria por atendente
Guardrails por padrão léxico, não por classificador semântico	Um ataque redigido de forma criativa o bastante passa pelo filtro	O que sustenta a segurança é a arquitetura: não existe caminho de código que leia dados de outro titular. O filtro reduz ruído e dá visibilidade
Código do segundo fator devolvido na resposta da API	Só existe no protótipo, para exibir o SMS simulado	Envio real por SMS ou e-mail, removendo o campo da resposta
Banco local sem criptografia em repouso	Leitura do arquivo expõe a base, ainda que o CPF esteja mascarado desde a origem	Banco gerenciado com criptografia em repouso e gestão de chaves
Sem expurgo automático de dados	Os registros ficam indefinidamente, ainda que a recuperação de memória seja limitada a 24 horas	Rotina de retenção e endpoint de exclusão a pedido do titular
Provedor de linguagem simulado	As respostas vêm de modelos de texto por intenção, linha e persona, não de um modelo generativo	Trocar apenas esse módulo: o núcleo de orquestração não muda

7. Como reproduzir

Pré-requisito único: Node.js 22.5 ou superior. Sem Docker, sem banco externo, sem chave de API.

Terminal 1 — backend

```
cd clarointelligence-api
npm install
npm run seed
npm run server
```

Terminal 2 — frontend

```
cd clarointelligence
npm install
npm run dev
```

O painel abre em localhost:5173 e a API responde em localhost:3001. O comando de carga inicial apaga e recria a base: é também o que limpa as conversas de ensaio e devolve o painel ao estado de demonstração. Os 11 roteiros estão descritos em COMO_RODAR.md, com o que digitar em cada um.

Código-fonte, documentação técnica e histórico completo de alterações em
github.com/RaphaelBatizati/ClaroIntelligent_Challenge